

团队级 AI Coding 简明手册

v0.2 2026.06



团队级 AI Coding
简明手册

AI 编程思想发展过程

你也许实践了部分内容，但业界领袖给出了更精确的表述

每一次思想的变更是对前者的重新表述而非推翻



我们在这里

用具体问题描述我们的处境

01 需求衔接 如何用 AI 整理需求？

- 如何编写 AI 能理解的需求？
 - 使用什么格式能被 AI 理解？
 - 如何组织需求规格？
 - 如何传递原型图？
- 需求的模版长什么样？
 - 表达新建需求的模版？
 - 表达变更需求的模版？
-

02 开发实现 如何获得高质量的代码？

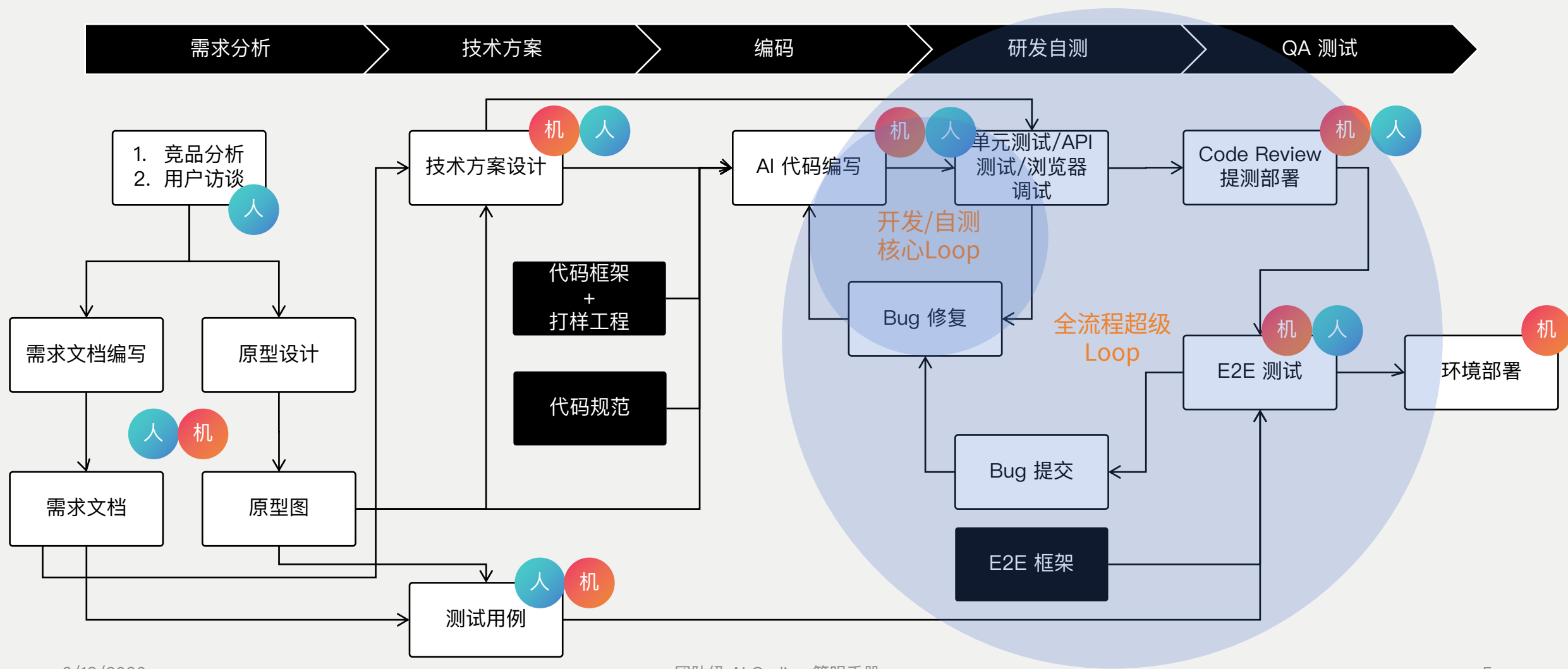
- 如何让 AI 理解上下文且按照要求执行？
- 如何让 AI 调用外部工具？
- 如何让 AI 生成一致的代码？
- 如何多任务同步开发？
- 如何进行调试？
-

03 测试验收 如何根据规格自动化测试？

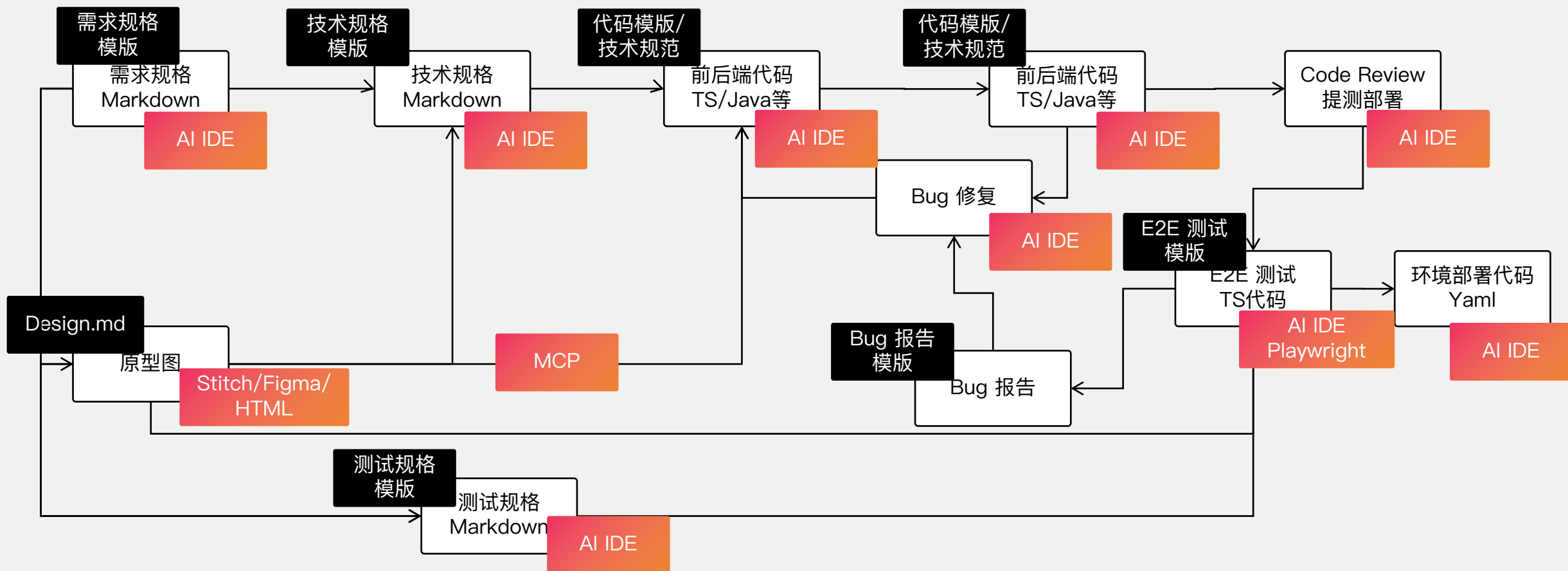
- AI 编程还需要什么测试策略？
- 如何让 AI 操作浏览器？
- 如何管理测试用例？
-

04 Agent as Code 如何把和 AI 协作的文件在代码仓库中合理组织，构建 AI 更友好的环境？

AI 赋能软件开发实现地图



AI 赋能软件开发工具体系



01 需求衔接

需求规格编写

Tips: BA 也应该工作在代码仓库中，可以让 AI 基于当前的**系统事实**构建新的需求。

AI 能理解的需求格式：Markdown/Excel 表格

关键要素：模块切分(增量/存量)+字段清单+原型链接+业务规则

📁 模块切分

- 先写业务背景，再切模块。一个模块 = 一个文档。
- 分解为增量需求和存量需求：
 - 增量需求 — 全新功能，需新建模块
 - 存量修改 — 对已有功能做变更

Tips: 增量建模块 · 存量标改动

≡ 字段清单

- 定义完整字段
- 标注：
 - 校验规则
 - UI 展示
 - 数据来源

字段名	类型	必填	默认值	描述
username	string	是	—	登录用户名
email	string	是	—	电子邮箱
role	enum	否	user	用户角色
status	boolean	否	true	是否启用

◇ 原型设计

- 使用 Figma / Stitch URL 或 HTML 原型嵌入需求文档。
 - Figma — 选中 Frame → Share → Copy link
 - Stitch — 选中设计稿 → 获取分享链接
- HTML 版本 — 用 HTML/CSS 做可交互原型，提交到代码仓库。

✓ 业务规则

- 条目化 + 正交分解，每条规则独立可测。

- R1 用户名唯一不可重复
- R2 邮箱格式正则校验
- R3 密码 ≥ 8 位含大小写
- R4 删除需二次确认 R5 超时自动登出 R6 管理员不可降级

原型图设计

◇ AI 设计软件输出

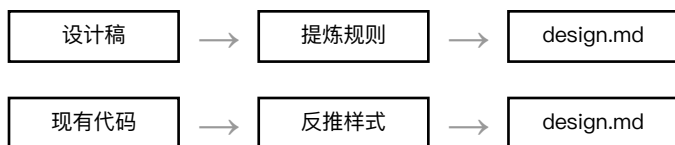
- 在设计工具中完成原型，通过 URL 链接嵌入需求文档。标注映射 — 在需求中注明对应模块和页面
 - Figma — 选中 Frame → Share → Copy link
 - Stitch — 选中设计稿 → 获取分享链接

📄 HTML + CSS 输出

- 用 HTML/CSS 做可交互原型，提交到代码仓库。
- 真实代码 — 直接使用项目组件库拼装
- 可交互 — 点击、跳转、表单输入均可演示
- 版本管理 — 随代码仓库一起管理和 Review

design.md 生成统一的视觉风格

- Design.md — 用 Markdown 约束 AI 生成的前端风格，确保输出与项目组件库一致。
- 两种产生方式：



Design.md 参考示例 ➡

更多参考 [https:// getdesign.md](https://getdesign.md)

```
# Design System
## Colors
- primary: #1a1a1a
- background: #ffffff
- text: #333333
## Typography
- font: -apple-system, sans-serif
- h1: 1.5rem / 700
- body: 0.875rem / 400
## Spacing
- unit: 8px (base)
```


02 开发实现

如何让 AI 听话-规则、规格、技能

规则 Rules



约束 AI 行为的边界，定义能做什么、不能做什么。

Java 代码风格 Rules

1. 使用 Java 17 语法，不使用已废弃 API。
2. 类名使用大驼峰（PascalCase），方法名和变量名使用小驼峰（camelCase）。
3. 常量使用全大写下划线分隔（UPPER_SNAKE_CASE）。
4. 单行代码不超过 120 字符，超过则换行。
5. 大括号采用 Egyptian 风格（左括号不换行）。
6. 禁止使用 `@Autowired` 字段注入，改用构造器注入。

规格 Specification



把模糊的需求收敛为结构化的内容，甚至通过领域特定语言 DSL 描述。

“请做一个登陆注册页面，需要能实现邮箱验证。”

提示词

作为一个新用户或现有用户，我想要一个同时支持注册和登录的页面，并且在注册时能通过邮箱完成身份验证，以便安全地创建账户并登录系统。

验收标准 (AC)：

1. 用户可以输入邮箱、密码等必要信息进行注册。注册提交后，系统向用户邮箱发送验证链接（或验证码）。用户点击链接（或输入验证码）后，账户状态变为“已验证”。
2. 已验证用户可使用邮箱和密码登录系统。未验证邮箱的用户尝试登录时，系统提示“请先完成邮箱验证”。
3. 邮箱格式不正确或已被注册时，系统给出明确提示。验证链接过期或无效时，支持重新发送验证邮件。

规格文档

技能 Skill



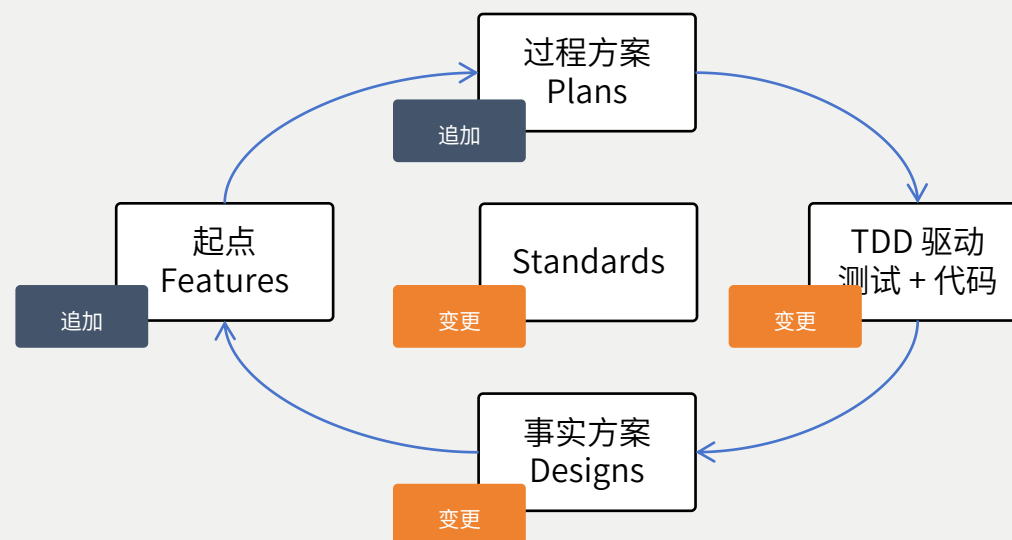
通过打包提示词、脚本、模版，进一步精确拓展 AI 的能力。

Skill Folder Structure

```
your-skill-name/
├── SKILL.md           # Main instructions for Claude
├── resources/         # Supporting files (optional)
│   ├── examples/
│   ├── templates/
│   └── data/
├── scripts/           # Executable code (optional)
└── helper.py
```

如何让 AI 听话-构建 AI 上下文体系

使用完整的文档来描述上下文和约束体系，区分过程方案和事实方案，减少维护成本。



Plan/Go 模式下的规格流转

追加: 每次创建一个新的文件，细微修改不再更新，作为过程方案。

变更: 每次以最终态更新，作为下一次 Plan 的事实依据。

如何让 AI 调用外部工具？

⇒ MCP

Model Context Protocol — AI 与外部系统通信的标准协议。

- 标准化接口 — 数据库、API、文件系统等统一接入
- 双向通信 — AI 读/写外部资源，实时响应
- 即插即用 — 配置即可用，无需写胶水代码

```
# MCP 配置示例
{
  "mcpServers": {
    "database": {
      "command": "node",
      "args": ["mcp-server-sqlite"]
    }
  }
}
```

☰ Skills

将开源工具用脚本封装成 AI 可直接调用的能力单元。

- 脚本封装 — 将 CLI / API 封装为 Skill
- 触发词驱动 — 用户输入触发词即执行
- 可复用 — 团队共享常用工具 Skill

```
# Commit & Push Skill
## 触发词: commit, push
## 执行
git add -A
git commit -m "{message}"
git push
## 规则
- 提交前检查 lint
- 信息遵循 conventional commit
```

◇ Commands

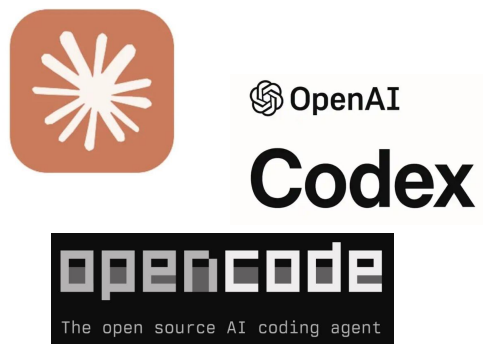
通过 CLI 命令让 AI 直接调用外部工具，最直接的方式。

- 即时执行 — AI 生成命令并执行
- 管道组合 — 多个命令串联完成复杂任务
- 权限可控 — 可配置允许/禁止的命令

```
# AI 通过 CLI 调用工具
> 部署到生产环境

npm run build
npx vercel --prod
```

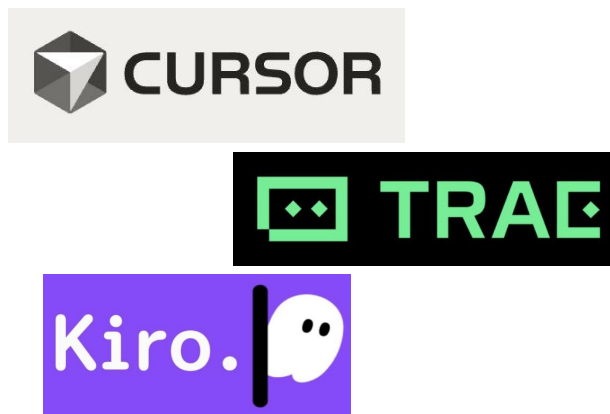
用什么 AI 编程工具？



命令行类

习惯使用命令行，AI 优先的开发模式。

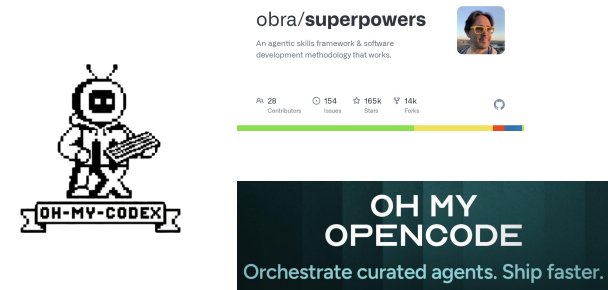
- Claude Code
- Codex CLI
- OpenCode



AI IDE

对代码掌控感更强，需要人介入程度更深。

- Cursor
- Kiro
- Trae



增强插件

需要预制的一系列辅助工具、脚本、模版。

- Superpower
- oh-my-opencode
- oh-my-codex

用什么 SDD 框架比较好？

Spec = 单一事实来源（SSoT），甚至可以直接生成代码。其实在非 AI 时代，也是这样工作的，只不过 Spec 在很多人的“大脑”中。使用 Spec 驱动开发的模式就是 SDD。

维度	BMAD	Spec Kit	OpenSpec	Kiro
方法定位	企业级 SDD 操作系统	工程化 Spec workflow	轻量 Spec 层	IDE 原生 SDD
核心理念	Spec = 治理体系 + 多 Agent 编排	Spec = 开发入口 + Git 生命周期	Spec = 变更单元（持续演化）	Spec = 可执行源（代码与测试）
Spec 生命周期	全生命周期（PRD → 架构 → 实现）	与分支绑定（短生命周期）	长期存在（持续演化）	持续驱动（开发时实时参与）
Spec 粒度	大（系统 / 模块级）	中（Feature级）	小（变更 / Patch级）	中（Feature + 行为）
可执行能力说明	通过流程驱动实现，不直接运行	驱动开发流程，不参与运行	类 Prompt，可生成代码但不可验证	可生成代码、测试并自动校验
行为表达结构化	强规范	部分结构化	轻量 Given/When/Then	严格结构（EARS）
流程控制	强（阶段 + 审批 + Agent）	中（Plan → Spec → Tasks）	弱（自由演化）	强（Spec → 生成 → 验证闭环）
Agent 参与方式	多 Agent 分工	辅助执行	Prompt 级辅助	IDE 内主导执行
自动验证能力	无（需外部测试）	无	弱（依赖外部）	强（内建验证机制）
与代码关系	间接（通过流程）	半耦合（驱动实现）	弱耦合	强耦合（直接生成与校验）
适用场景	大型系统 / 多团队 / 强治理	新项目（0→1）	存量项目 / 快速迭代	小团队 / 高自动化开发
失控风险	低	中	高	中

一个新星：Superpower

Superpower 提供了一些内置的 skills，方便开箱即用，缺点是不太方便精细调整。

1. **brainstorming** - Activates before writing code. Refines rough ideas through questions, explores alternatives, presents design in sections for validation. Saves design document.
2. **using-git-worktrees** - Activates after design approval. Creates isolated workspace on new branch, runs project setup, verifies clean test baseline.
3. **writing-plans** - Activates with approved design. Breaks work into bite-sized tasks (2-5 minutes each). Every task has exact file paths, complete code, verification steps.
4. **subagent-driven-development** or **executing-plans** - Activates with plan. Dispatches fresh subagent per task with two-stage review (spec compliance, then code quality), or executes in batches with human checkpoints.
5. **test-driven-development** - Activates during implementation. Enforces RED-GREEN-REFACTOR: write failing test, watch it fail, write minimal code, watch it pass, commit. Deletes code written before tests.
6. **requesting-code-review** - Activates between tasks. Reviews against plan, reports issues by severity. Critical issues block progress.
7. **finishing-a-development-branch** - Activates when tasks complete. Verifies tests, presents options (merge/PR/keep/discard), cleans up worktree.

技术规格如何编写？

技术规格使用 DSL 驱动包含：领域模型、数据库、API、时序图、专题设计

模块	表达标准	格式	产出内容	约束 / 规范
领域模型	PlantUML/Smart Domain	.puml	实体、属性、关系 (ER / 类图)	必须表达实体关系；字段需与数据库一致；标注聚合关系
数据库	SQL DDL/DBML/Flaywa 脚本	.sql	表结构、索引、约束	必须可执行；包含索引和外键；字段与领域模型一致
API	OpenAPI 3.x	.yaml / .json	接口定义、请求/响应结构	必须定义 schema；禁止只写接口说明；字段与模型一致
时序图	PlantUML	.puml	调用流程、系统交互	必须反映真实调用链；建议包含异常分支 (alt)
专题设计	Markdown	.md	架构策略（权限/事务/缓存等）	只写“策略与规则”，不重复 API/DDL；强调设计决策

```
# 技术规格文档 (Technical Specification)

## 1. 文档说明

### 1.1 背景
(描述业务背景与问题)

### 1.2 目标
* 实现能力:
* 技术边界:

---

## 2. 领域模型 (PlantUML)

```plantuml
@startuml
' 在此定义实体、属性、关系

entity User {
+id: string
+name: string
}

entity Order {
+id: string
+userId: string
}

User ||--o{ Order : places
@enduml
```

---

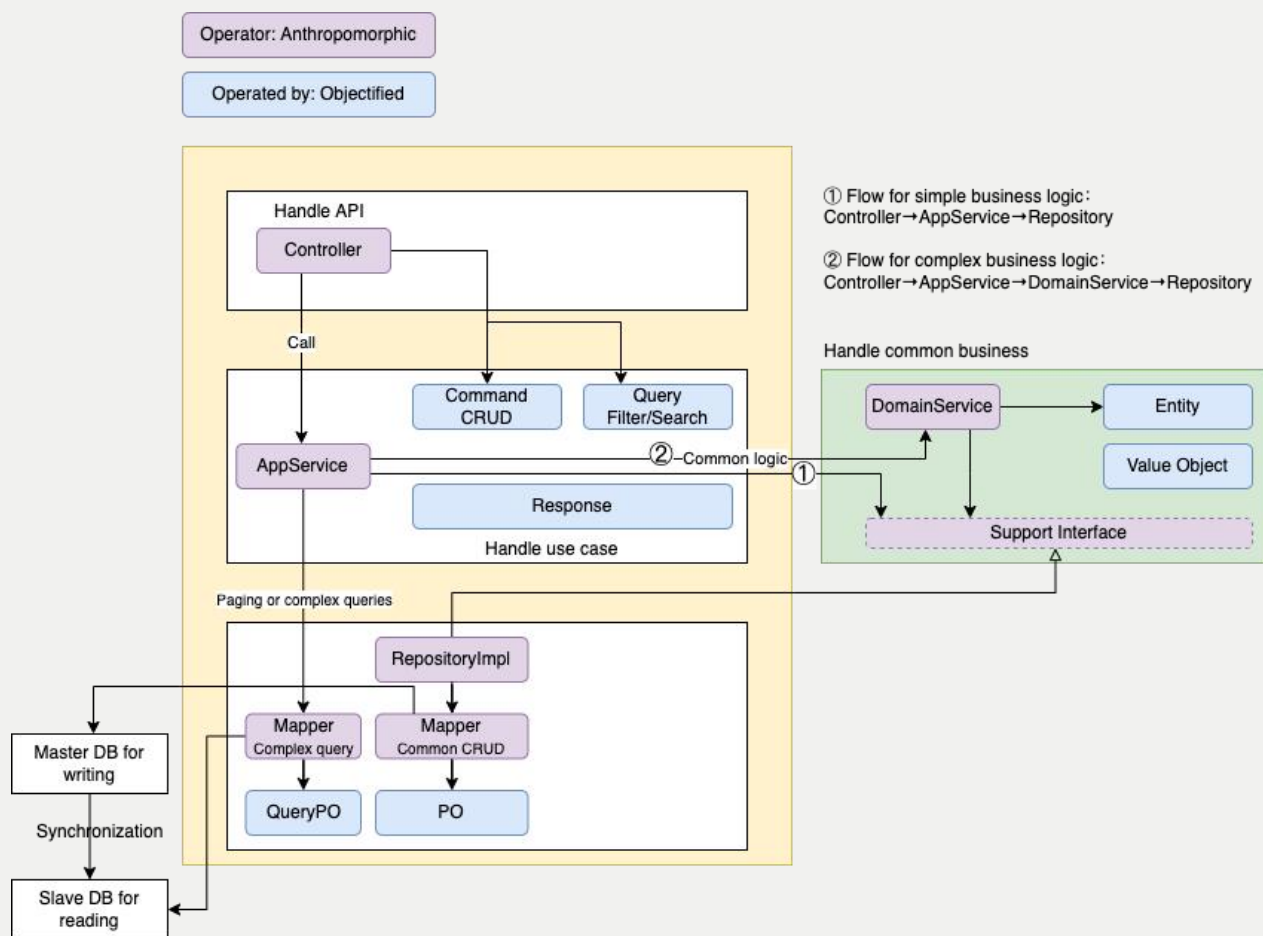
## 3. 数据库设计 (SQL DDL)

```sql
-- 在此定义表结构 (必须可执行)

CREATE TABLE user (
 id VARCHAR(64) PRIMARY KEY,
 name VARCHAR(255) NOT NULL
);

CREATE TABLE `order` (
 id VARCHAR(64) PRIMARY KEY,
 user_id VARCHAR(64) NOT NULL
);
```
```


打样工程：如何让 AI 抄作业？



原则：约定大于配置、编排逻辑和原子能力分离、操作者和被操作对象分离。

左图为一套适用于常见 CQRUD 操作的工序：

- 实现简单 CRUD，操作工序：Controller + Command → AppService → Repository → Response 对象
- 实现复杂或者有复用逻辑的 CRUD，操作工序：Controller + Command → AppService → DomainService → Repository → Response 对象
- 实现复杂的查询操作工序：Controller + Query 对象 → AppService → QueryPO（或者复用 PO）→ Response 对象

参考代码: <https://github.com/domain-driven-design/ddd-microservices>

如何多任务同步开发？

使用 git 的 Worktree 功能同时把一个仓库的多个分支映射到不同文件夹。

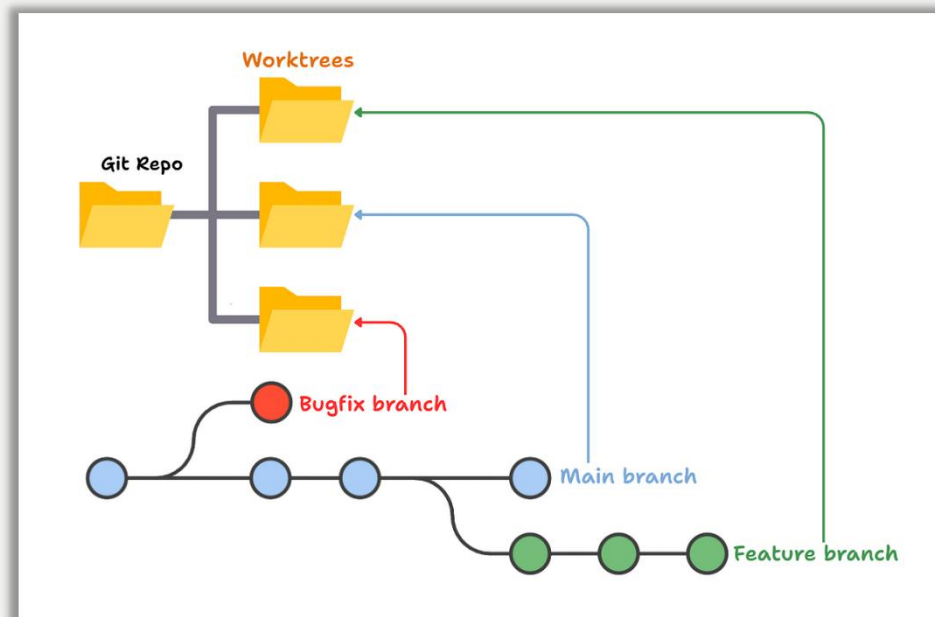
Claude Code、Codex、Cursor 等已经自动支持使用 subagent 创建 Worktree，加速开发。

```
// 基于已有分支创建
git worktree add ../feature-login feature/login

// 创建新分支 + 工作区（最常用）
git worktree add -b feature-payment ../feature-payment

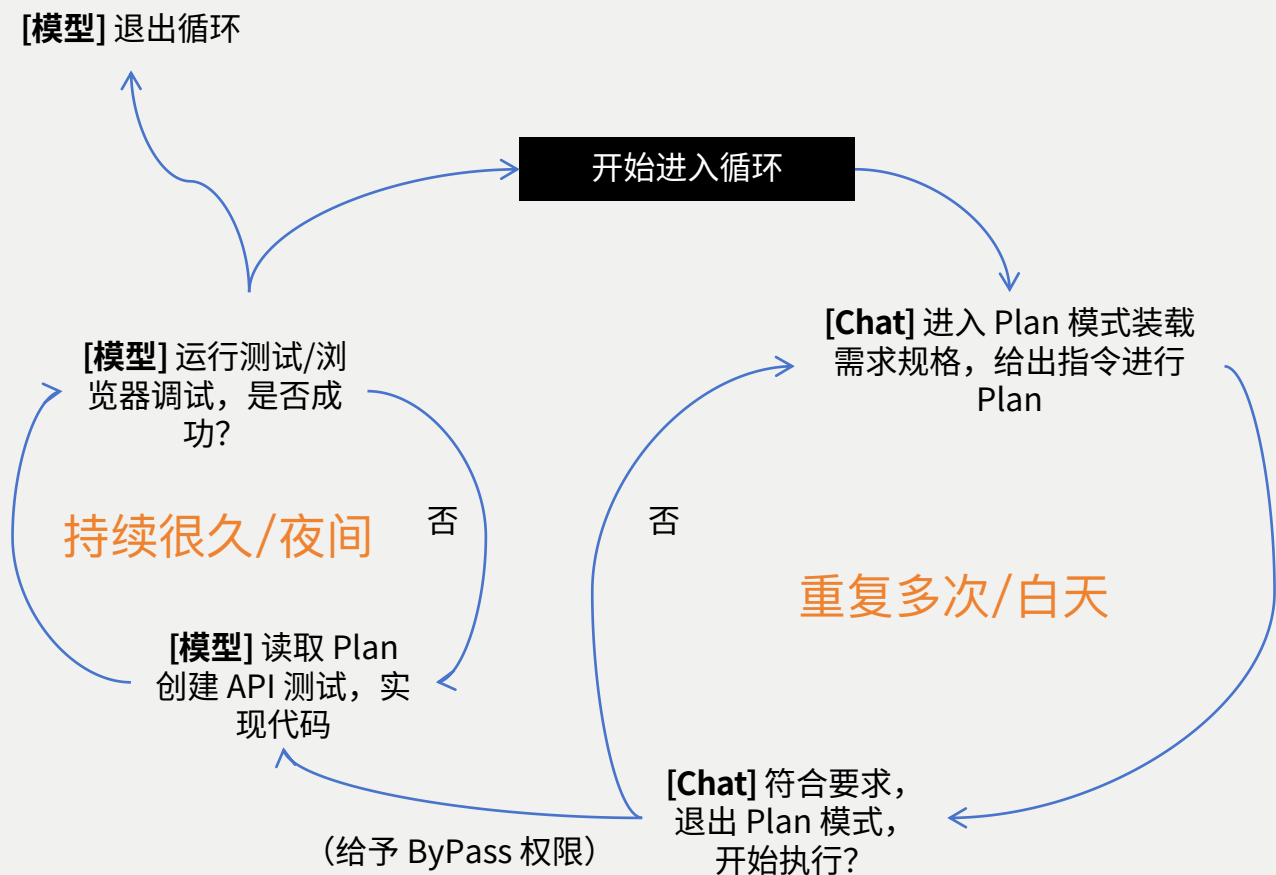
// 查看所有 worktree
git worktree list

// 删除工作区
git worktree remove ../feature-login
```



完整的核心 Loop 过程（研发自测）

通过 TDD/浏览器调试（前端），AI 自治运行



核心思想：

- Plan 阶段尽可能锁定上下文到 Plan 中，关闭所有的开放性问题。
- Plan 中记录实现状态，让任务可以分批完成或者重试。
- 需要给出确定性的验收方式：例如通过所有的 API 测试，让 AI 能建立自我修复标准。

Loop 规则（API 为例，放到 Agent.md 作为开发流程要求）：

- **Plan 等待确认：**在 docs/plans 下根据模版创建 Plan
- **编写 API 测试：**根据 Plan 实现 API 测试，运行成功并断言失败。
- **编写实现：**编写单元测试 + 实现代码，通过编译。
- **运行 API 测试：**运行测试，并修复失败的测试和其他错误。

03 测试验收

AI 辅助下的测试策略是什么？

| 测试类型 | 测试对象 | 工具 (含 AI) | 负责人 (人 / Agent) |
|-------------|--------------------------------|--|-----------------------------|
| Lint 代码扫描 | 代码风格、代码逻辑和片段 | TS Lint/SonarQube | 程序 |
| Code Review | 代码逻辑、架构设计、规范 (复杂度/安全/可维护性) | GitHub Copilot、Claude Code、CodeRabbit、GitHub PR | 开发者 + AI Review Agent |
| 单元测试 | 函数/类行为、边界条件、异常处理 | JUnit、pytest、Jest + AI 生成测试代码 (Copilot / Claude) | 开发者 + Test Generation Agent |
| API 测试 | 接口契约 (Schema)、输入输出、鉴权、错误码、业务规则 | Karate (主)、RestAssured (补充复杂逻辑) + AI 生成测试用例/数据 | QA + 后端开发 + API Test Agent |
| E2E 测试 | 用户关键路径 (登录/下单/支付)、跨系统流程、前后端联动 | Playwright + AI 生成脚本 / 自愈 (selector 修复、步骤补全) | QA + 自动化测试工程师 + E2E Agent |

AI 如何操作浏览器？

从浏览器 API 到视觉操作，选择合适工具，权衡速度和 Token 消耗

| 维度 | OpenWright (Playwright MCP) | Chrome DevTools MCP | Browser Use | Computer Use |
|----------|---|--|---|--|
| 原理 | 通过 Playwright 访问浏览器 Accessibility Tree，获取结构化 DOM 快照，AI 基于节点操作 | 通过 Chrome DevTools Protocol 直接与浏览器引擎通信，暴露 CDP 全部能力（DOM/网络/控制台） | Python 框架 + Playwright，AI 自主决策循环；支持 DOM 和截图双模 | AI 截取屏幕截图 → 视觉识别元素 → 输出坐标/按键操作 → 沙箱执行，OS 级通用 |
| 抽象层 | Accessibility Tree
结构化 DOM 快照 | CDP Protocol
DevTools 协议原生 | DOM + 截图
视觉 + 结构化混合 | 截图 + 坐标
OS 级视觉理解 |
| 后端引擎 | Playwright | Puppeteer + CDP | Playwright + Python | Anthropic API + 沙箱 |
| 速度 | 快 ~0.9s/步 | 中 ~1.2s/步 | 中 ~1.5s/步 | 慢 0.8-2s/步 |
| Token 消耗 | 高
截图 + 结构全传 | 中
按需取数据 | 极低
CLI 模式 ~75 tok/步 | 高
截图编码开销大 |
| JS 重页面 | 中 — DOM 可读 | 中 — CDP 可取 | 中 — 视觉兜底 | 高 — 视觉理解 |
| 跨应用操作 | 仅浏览器 | 仅浏览器 | 仅浏览器 | 全桌面 |

如何用测试用例生成E2E测试？

Browser Use 探索

AI 自主遍历页面，记录操作轨迹、选择器、页面状态，输出 Playwright 脚本雏形。

消耗 Token

Playwright E2E 固化

将探索结果转为 Playwright 测试脚本，加入 CI，零 Token 可重复运行。

无需 Token

截图视觉兜底

Playwright 覆盖不到的视觉场景（布局、动效、图表），重回截图判断。

消耗 Token

Browser Use 探索 → Playwright 脚本

1. "用 Browser Use 打开 /login，走一遍登录流程"
2. 提取 locator、action、assertion
3. 生成 test-login.spec.ts，加入 CI
4. 后续运行 0 Token

```
test('login', async ({ page }) => {  
  await page.goto('/login');  
  await page.fill('[name="email"]', 'test@x.com');  
  await page.fill('[name="password"]', '123456');  
  await page.click('button[type="submit"]');  
  await expect(page.locator('.welcome')).toBeVisible();  
});
```

Playwright 拓展—截图+大模型

- Playwright Accessibility Tree 覆盖不到的场景：
 - 视觉回归 — 布局偏移、样式异常
 - 动效/动画 — 加载态、过渡动画完成
 - Canvas/WebGL — 图表渲染、游戏画面

// Playwright 可视化测试 helper

页面访问

截图

断言

图片识别

Playwright E2E 示例

使用 Playwright + AI 生成的脚本实现。

```
import { test, expect, request } from '@playwright/test';

test.describe('User Login Flow', () => {

  test('login success and dashboard visible', async ({ page }) => {

    // 1. 打开登录页
    await page.goto('https://example.com/login');

    // 2. 填写表单
    await page.getByPlaceholder('Email').fill('test@example.com');
    await page.getByPlaceholder('Password').fill('123456');

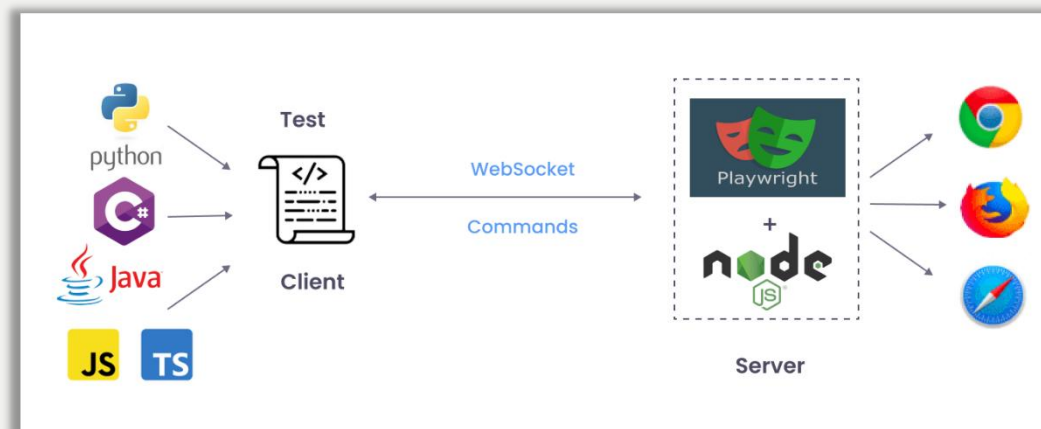
    // 3. 提交
    await page.getByRole('button', { name: 'Login' }).click();

    // 4. 等待跳转 & 断言
    await expect(page).toHaveURL(/dashboard/);
    await expect(page.getByText('Welcome')).toBeVisible();

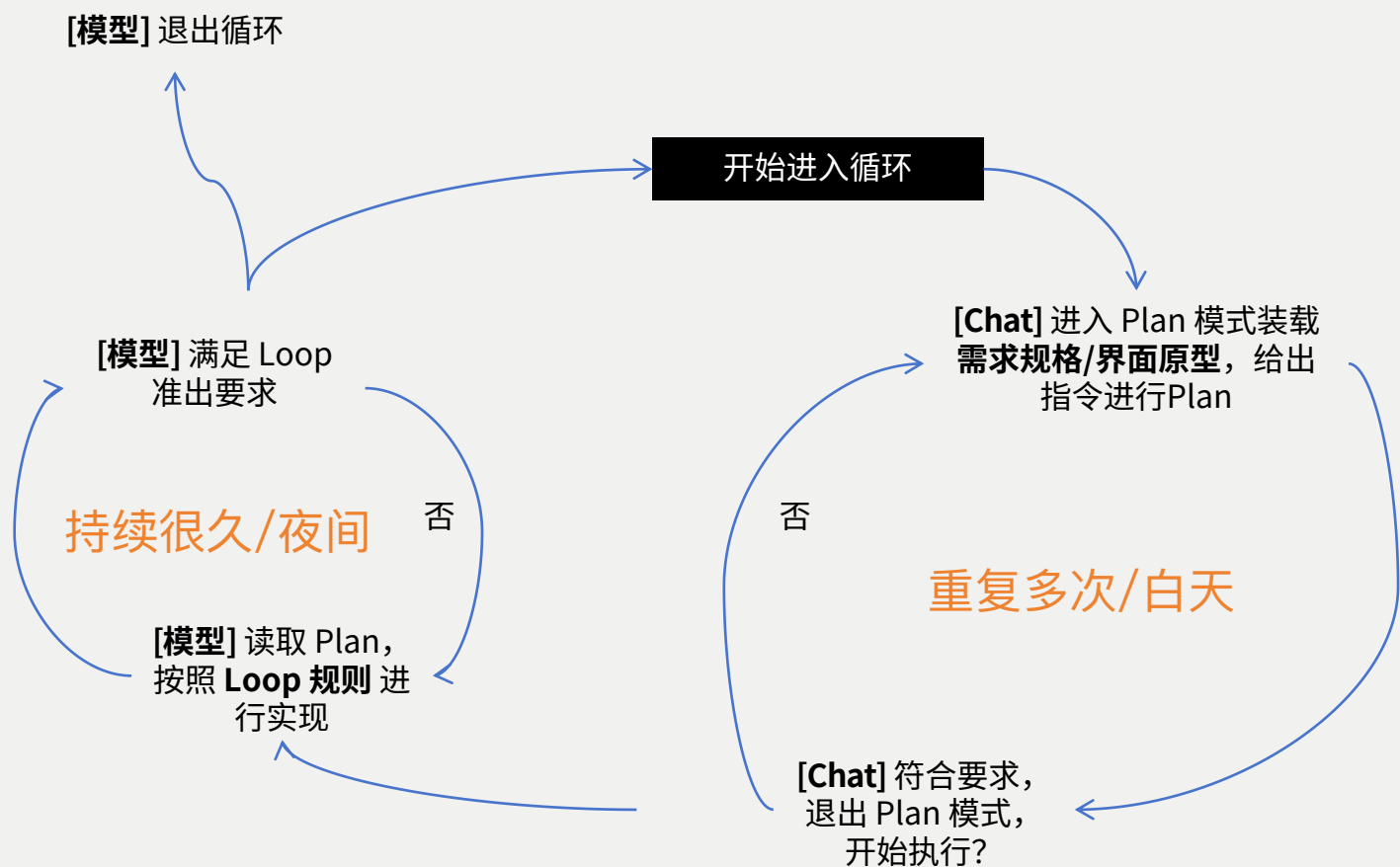
  });

  test('login via API + set session (更稳定)', async ({ page, request }) => {
  });

});
```



超级 Loop (E2E Loop)



核心思想：

- 通过 E2E 测试纳入更大的 Loop 范围。

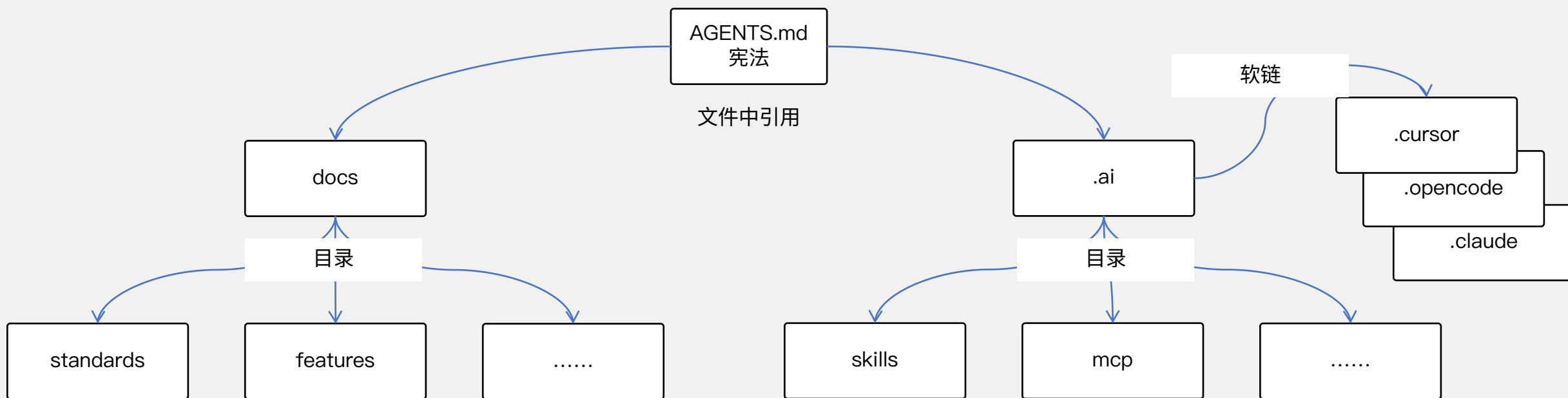
Loop 规则（放到 Agent.md 作为开发流程要求）：

- Plan 等待确认：**在 docs/plans 下根据模版创建 Plan。
- 编写测试用例：**在 docs/test-cases 下编写 E2E 测试用例。
- 编写 API 测试：**根据 Plan 实现 API 测试，运行成功并断言失败。
- 编写实现：**编写单元测试 + 实现代码，通过编译。
- 运行 API 测试：**运行测试，并修复失败的测试和其他错误。
- 浏览器调试：**使用 Playwright 调试，确认前端视觉效果和交互功能正常。
- 编写 E2E 测试：**编写 Playwright E2E 测试，覆盖本轮需求的测试场景。

04 Agent as Code

AI 工程文件管理

同时让 Claude/OpenCode 等工具支持相关 Harness 文件
我尝试过很多种方式，最简单的是使用 AGENTS.md 构建引用关系。



附录&参考资料

关于 AI 编程实践的心得

放弃“开箱即用”的框架，每个项目需要构建自己 Harness 工具和文档

“少量取用，大量实践”是更好的策略

期待您的反馈，以便我持续补充和迭代这份文档！

参考资料

- <https://openai.com/index/harness-engineering/>
- <https://martinfowler.com/articles/harness-engineering.html>
- <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- 从 Rule、Spec 到 Harness: AI Coding 的渐进式建设路径 <https://mp.weixin.qq.com/s/UCh2bPzMZjNBMBCCJysuNw>
- 让 AI 乖乖听话的几个 Rules https://mp.weixin.qq.com/s/FXNzk_y2Z2h8BVe62uEn_A
- Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- harness-engineering-in-ai-coding <https://github.com/2361485765/harness-engineering-in-ai-coding>
- <https://github.com/entropy-cloud/attractor-guided-engineering-template>
- <https://developers.openai.com/api/docs/guides/tools-computer-use>